

Iteration, Recursion, and Big-O

DAEN 300

Alexander Abuabara

Fall 2026



1. Reliable Mini Data Pipeline
2. Process Numerical Data
3. The Great M&M Count
4. Preparing Data for a Simple Machine

1. The philosophical difference between using a tool versus living inside a system?
2. AI functions more like an active system or environment than a passive, neutral tool that you fully control.
3. Calling AI "just a tool" is misleading.
4. AI is not "yours"
- 5.

Learning Goals

- Explain the difference between an iterative and recursive approach;
- Sort a small list of music artists by play count;
- Describe why algorithms take more time as data gets larger;
- Use basic Big-O notation to compare algorithm growth;
- Connect theoretical complexity with measured time using `timeit`.

Our Small Music Dataset

We will rank artists by total plays.

```
music_data = [  
    ("Taylor Swift", 950),  
    ("Bad Bunny", 870),  
    ("Billie Eilish", 620),  
    ("Drake", 810),  
    ("SZA", 590),  
    ("The Weeknd", 780),  
    ("Olivia Rodrigo", 540),  
    ("Bruno Mars", 690),  
    ("Kendrick Lamar", 730),  
    ("Sabrina Carpenter", 660)  
]
```

Goal: highest play count gets rank 1.

What Does Ranking Mean?

Rank	Artist	Plays
1	Taylor Swift	950
2	Bad Bunny	870
3	Drake	810
4	The Weeknd	780
5	Kendrick Lamar	730

Sorting is the process. Ranking is the labeled result.

Sort by play count descending $\Rightarrow$ assign rank numbers

Two Ways to Think About Repetition

Iteration

- repeats with loops;
- often uses `for` or `while`;
- usually easy to trace step by step.

Recursion

- repeats by calling the same function again;
- solves a smaller version of the problem;
- must have a stopping rule.

Both can solve the same problem.

The difference is how the repeated work is organized.

Small Iterative Example: Counting Plays

Iteration uses a loop.

```
plays = [950, 870, 620]

total = 0
for play_count in plays:
    total = total + play_count

print(total)
```

The loop visits each value once.

$$950 + 870 + 620 = 2440$$

Small Recursive Example: Counting Plays

Recursion breaks the list into one item plus the rest.

```
def total_plays(values):  
    if len(values) == 0: # base case  
        return 0  
    return values[0] + total_plays(values[1:])  
  
print(total_plays([950, 870, 620]))
```

The function calls itself with a smaller list each time.

$$[950, 870, 620] \rightarrow [870, 620] \rightarrow [620] \rightarrow []$$

The Most Important Rule of Recursion

Every recursive function needs:

Base Case + Recursive Case

Part	Meaning
Base case	The condition that stops the function
Recursive case	The part where the function calls itself on a smaller problem

Without a base case, the function keeps calling itself until Python raises an error.

Iterative Sorting Idea: Neighbor Swaps

One simple sorting strategy compares neighboring records.

```
if result[j][1] < result[j + 1][1]:  
    result[j], result[j + 1] = result[j + 1], result[j]
```

Example comparison:

Artist A	Plays	Compare	Artist B	Plays
Billie Eilish	620	<	Drake	810

Because 620 is less than 810, swap them.

Iterative Ranking Function

```
def rank_iterative(data):  
    result = data.copy()  
    n = len(result)  
  
    for i in range(n):  
        for j in range(0, n - i - 1):  
            if result[j][1] < result[j + 1][1]:  
                result[j], result[j + 1] = result[j + 1],  
                    result[j]  
  
    return result
```

This uses nested loops.

Outer loop $\times$ inner loop $\approx n \times n$

Recursive Ranking Function

Here is one beginner-friendly recursive sorting approach.

```
def rank_recursive(data):  
    result = data.copy()  
  
    def bubble_pass(end):  
        if end <= 1: # base case  
            return  
  
        for j in range(end - 1):  
            if result[j][1] < result[j + 1][1]:  
                result[j], result[j + 1] = result[j + 1],  
                    result[j]  
  
        bubble_pass(end - 1) # recursive case  
  
    bubble_pass(len(result))  
    return result
```

Comparing the Two Approaches

Question	Iterative	Recursive
How does it repeat?	Uses loops	Calls itself
Stopping rule	Loop range ends	Base case is reached
Easy for beginners?	Often yes	Sometimes less obvious
Can solve sorting?	Yes	Yes
Can have same Big-O?	Yes	Yes

Why Do We Need Big-O?

Small examples can hide big problems.

Number of Artists n	Approximate n^2 Work
10	100
20	400
50	2,500
100	10,000
1,000	1,000,000

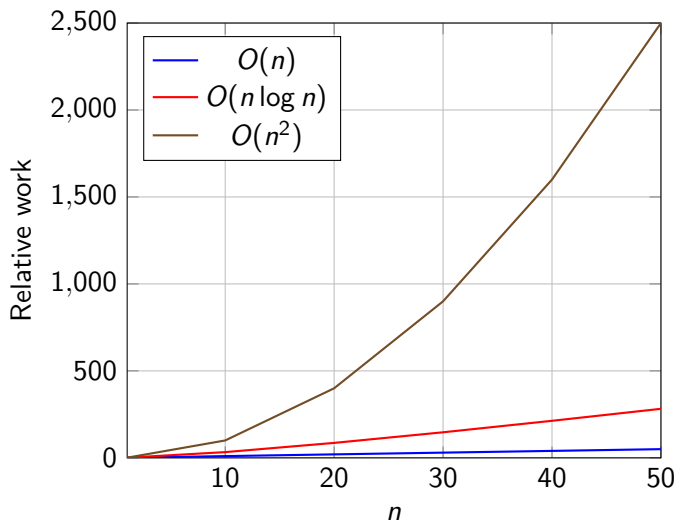
Big-O helps us describe how work grows as data grows.

Common Big-O Patterns

Big-O	Name	Example Idea
$O(1)$	Constant	Get the first item
$O(n)$	Linear	Visit every artist once
$O(n \log n)$	Efficient sorting	Python's built-in sort
$O(n^2)$	Quadratic	Compare many pairs

Big-O ignores small details and focuses on growth.

Growth of Common Big-O Classes



Big-O for Our Ranking Algorithms

Both beginner sorting versions use repeated comparisons.

Implementation	Repetition Style	Approximate Big-O
Iterative ranking	Nested loops	$O(n^2)$
Recursive ranking	Recursive passes	$O(n^2)$
Python sorted()	Built-in optimized sort	$O(n \log n)$

Note: recursion does not automatically make an algorithm faster.

Measuring Time with timeit

Theoretical complexity is useful, but we can also measure runtime.

```
import timeit

iterations = 1000

iterative_time = timeit.timeit(
    lambda: rank_iterative(music_data),
    number=iterations
)

recursive_time = timeit.timeit(
    lambda: rank_recursive(music_data),
    number=iterations
)

print(iterative_time / iterations)
print(recursive_time / iterations)
```

Timing Results Table Template

Complete this table with your own results!

Implementation	Runs	Total Time	Avg. Time per Run
Iterative	1000		
Recursive	1000		
Python sorted()	1000		

Note that measured results may vary by computer, Python version, and background activity(ies).

How to Interpret the Results

A good comparison should include both theory and measurement.

Evidence Type	What It Tells Us
Big-O	How runtime grows as data gets larger
<code>timeit</code>	How fast the code was on one machine
Readability	How easy the code is to understand and maintain
Correctness	Whether the output ranking is right

Takeaways

- Iteration repeats work using loops.
- Recursion repeats work using function calls.
- Recursive functions need a base case.
- Big-O describes growth, not exact seconds.
- Two algorithms can have the same Big-O but different measured runtimes.
- Built-in Python tools are often preferred in professional code because they are tested and optimized.

Question for after the lab assignment

Suppose the music dataset grows from 10 artists to 10,000 artists.

Would you still use the beginner sorting algorithm from this lesson?

Explain your answer using:

Correctness + Big-O + <code>timeit</code> + Readability
